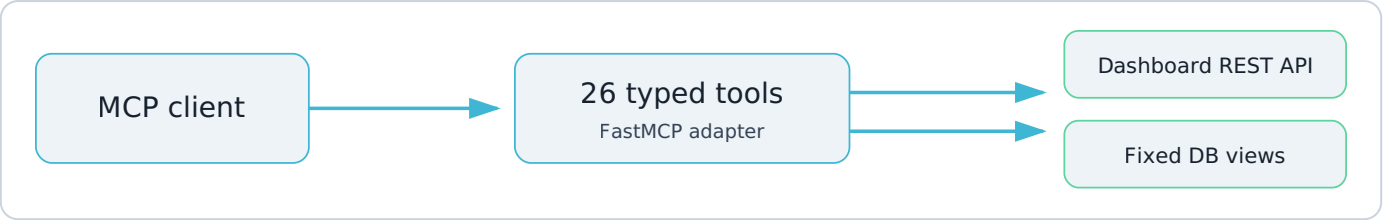
solarinet_mcp

An MCP server that gives an LLM agent eyes — and a careful set of hands — on a SolariNet deployment.

This server exposes three surfaces of the SolariNet monitoring system to an agent: the read-only dashboard API (§11.2), the operator control plane (survey / config push), and fixed read-only database views over the MariaDB schema (§10). It is written in **Python with FastMCP** — no Node, per project policy — and ships with unit tests and a verifiable evaluation suite.



> **Release boundary.** This candidate exposes published dashboard API operations and fixed database views. Generic caller-supplied SQL is not exposed as a tool. Unit tests use injected API and DB fixtures; run the project integration suite separately to verify deployment behavior.

Install

```
cd integrations/mcp
pip install -r requirements.txt        # or: pip install -e .
python -m solarinet_mcp --check        # prints config + the 26 tools, no server start
```

Configure (environment only — no secrets in files)

VARIABLE	PURPOSE	DEFAULT
`SOLARINET_API_BASE`	Dashboard API base URL, e.g. `https://c2.akoria.net`	(unset)
`SOLARINET_API_TOKEN`	Bearer token; required for control POSTs (operator role)	(unset)
`SOLARINET_API_TIMEOUT`	Per-request timeout (seconds)	30
`SOLARINET_API_VERIFY_TLS`	`0` disables TLS verify (lab only)	1
`SOLARINET_DB_HOST/PORT/NAME`	MariaDB coordinates	127.0.0.1 / 3306 / solarinet
`SOLARINET_DB_TIMEOUT`	MariaDB connect/read timeout (seconds)	10
`SOLARINET_DB_USER`	DB user — **use a GRANT SELECT-only account**	(unset)

VARIABLE	PURPOSE	DEFAULT
`SOLARINET_DB_PASS`	DB password	(unset)
`SOLARINET_MCP_TRANSPORT`	`stdio` or `streamable_http`	stdio
`SOLARINET_MCP_PORT`	Streamable HTTP listener port (must be 1–65535)	8765

Tools (26)

Read — REST `read-only`

TOOL	MAPS TO
`solarinet_list_nodes`	GET /api/nodes
`solarinet_get_node`	GET /api/nodes/{id}
`solarinet_get_node_history`	GET /api/nodes/{id}/history
`solarinet_list_probes`	GET /api/probes
`solarinet_get_probe_history`	GET /api/probes/{targetId}/history
`solarinet_list_alerts`	GET /api/alerts
`solarinet_get_topology`	GET /api/topology
`solarinet_server_status`	GET /api/server/status
`solarinet_list_discovery`	GET /api/discovery?status=
`solarinet_list_enrollments`	GET /api/enrollments?status=
`solarinet_list_builds`	GET /api/builds
`solarinet_list_segments`	GET /api/segments
`solarinet_list_netgear`	GET /api/netgear
`solarinet_get_topology_view`	GET /api/topology?view=monitoring|network
`solarinet_get_config`	GET /api/config

Control — REST `side-effecting`

TOOL	MAPS TO
`solarinet_trigger_survey`	POST /api/control/survey

TOOL	MAPS TO
`solarinet_push_config`	POST /api/control/config
`solarinet_adopt_discovery`	POST /api/discovery/{discId}/adopt
`solarinet_ignore_discovery`	POST /api/discovery/{discId}/ignore
`solarinet_reject_enrollment`	POST /api/enrollments/{enrId}/reject
`solarinet_provision`	POST /api/control/provision
`solarinet_approve_enrollment`	POST /api/enrollments/{enrId}/approve · operator + confirm
`solarinet_decommission`	POST /api/control/decommission · operator + confirm

DB views — fixed queries `read-only`

TOOL	WHAT IT DOES
`solarinet_db_node_summary`	Fleet roster from the `node` table
`solarinet_db_active_alerts`	Firing alerts joined to their rule
`solarinet_db_host_history`	hostHistory rows for one node

Safety model

- **Read vs. write are clearly separated** and annotated (`readOnlyHint` / `destructiveHint`) so clients can gate the eight control tools.
- **Database access uses fixed query functions.** MCP callers cannot supply SQL. The internal DB client still enforces read-only syntax, a table allow-list, and row caps; pair it with a SELECT-only DB grant for defense in depth.
- **No secrets in source.** All credentials come from the environment, matching SolariNet's own §13 convention.

Test & verify

```
cd integrations/mcp
python -m pytest -q             # MCP unit tests
python evals/check_answers.py    # derives and verifies all 10 answers through tools and fixtures
```

The evaluation questions live in `evals/evaluation.xml` ; they're read-only, independent, and verify to stable answers against the fixture in `evals/fixture.py` .

For module responsibilities, tool contracts, runtime configuration, and security boundaries, see the MCP Developer Reference.

Register with a client

Example stdio entry for an MCP client config:

```
{
  "mcpServers": {
    "solarinet": {
      "command": "python",
      "args": ["-m", "solarinet_mcp"],
      "env": {
        "SOLARINET_API_BASE": "https://c2.akoria.net",
        "SOLARINET_API_TOKEN": "…",
        "SOLARINET_DB_USER": "solari_ro",
        "SOLARINET_DB_PASS": "…"
      }
    }
  }
}
```

Part of the SolariNet Monitoring project · auxiliary tooling (Python/FastMCP). The C application and PHP dashboard remain the core deliverables; this server is a convenience integration over their published contracts.